

CWVSmix: Critical Window Variable Selection for Mixtures

CWVSmix_Example

[1] Simulate data for analysis:

- Setting the reproducibility seed and initializing packages for data simulation:

```
set.seed(2155)
library(CWVSmix)
library(boot) #Inverse logit transformation
```

- Setting the global data values:

```
n<-40000 #Sample size
m<-20 #Number of exposure time periods
q<-4 #Number of pollutants

x<-matrix(1,
nrow=n,
ncol=2) #Covariate design matrix
x[,2]<-scale(rnorm(n=n))

z_main<-matrix(rnorm(n=(n*m*q)),
nrow=n,
ncol=(m*q)) #Main effect design matrix

z_int<-matrix(0,
nrow=n,
ncol=(m*q*(q-1)/2)) #Interaction effect design matrix
counter<-1
for(j in 1:m){
  for(k in 1:(q-1)){
    for(l in (k+1):q){
      z_int[,counter]<-z_main[, (k + (j-1)*q)]*z_main[, (l + (j-1)*q)]
      counter<-counter +
        1
    }
  }
}

z<-matrix(0,
nrow=n,
ncol=(ncol(z_main) + ncol(z_int))) #Full set of exposures
for(j in 1:m){
  z[, (1+(j-1)*q*(q+1)/2):(j*q*(q+1)/2)]<-cbind(z_main[, (1+(j-1)*q):(j*q)],
    z_int[, (1+(j-1)*q*(q-1)/2):(j*q*(q-1)/2)])
}

for(j in 1:ncol(z)){
  z[,j]<-(z[,j] - median(z[,j]))/IQR(z[,j]) #Data standardization (interquartile range)
}
```

- Setting the values for the statistical model parameters:

```
beta_true<-c(-3.20, 0.10)
theta_true<-rep(0.60, times=m)
```

```

gamma_true<-c(rep(0, times=10),
rep(1, times=5),
rep(0, times=5))
alpha_true<-gamma_true*theta_true

lambda_true<-matrix(0,
nrow=m,
ncol=(q*(q+1)/2))

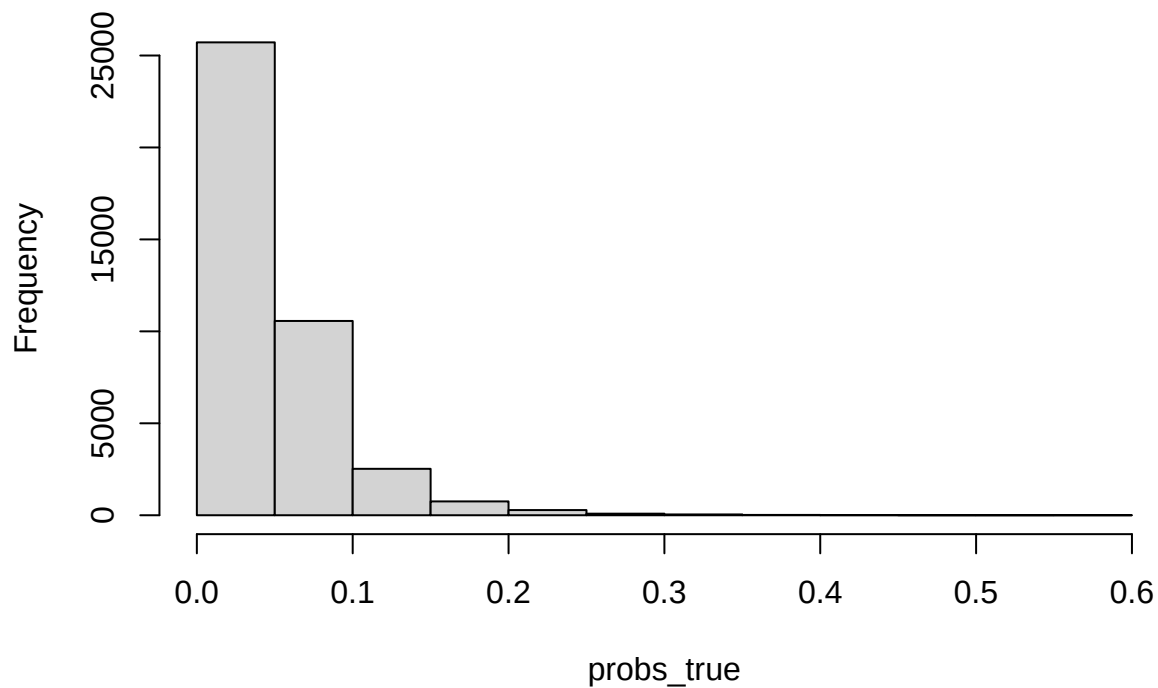
#Choosing pollutant/interaction weights on critical weeks
lambda_true[11,1]<-1
lambda_true[12,c(1,2,5)]<-c(0.30, 0.60, 0.10)
lambda_true[13,c(1,2,5)]<-c(0.30, 0.60, 0.10)
lambda_true[14,c(1,2,3)]<-c(0.30, 0.60, 0.10)
lambda_true[15,c(1,2,3,5,6)]<-c(0.40, 0.20, 0.10, 0.20, 0.10)

risk<-matrix(0,
nrow=n,
ncol=m)
for(j in 11:15){ #Critical weeks: 11-15
risk[,j]<-z[, (1 + (j-1)*q*(q+1)/2):(j*q*(q+1)/2)]%*%lambda_true[j,]
}

logit_p_true<-x%*%beta_true +
risk%*%alpha_true
probs_true<-inv.logit(logit_p_true)
hist(probs_true)

```

Histogram of probs_true



```
trials<-rep(1, times = n)
```

- Simulating the analysis dataset:

```
y<-rbinom(n=n,  
          size=trials,  
          prob=probs_true)
```

- Creating matched case-control data:

```
cases<-c(1:n)[y == 1]  
controls<-c(1:n)[y == 0][1:length(cases)]  
final_set<-c(cases,  
             controls)  
y<-y[final_set]  
  
x<-x[final_set,]  
  
z<-z[final_set,]  
  
pair_id<-rep(c(1:length(cases)), times = 2)
```

[2] Fit CWVS to identify/estimate critical windows of susceptibility:

```
results<-CWVSmix(mcmc_samples = 10000,  
                 p = (q*(q+1)/2), y = y, x = x, z = z,  
                 metrop_var_lambda_trans = rep(0.10, times=m),  
                 metrop_var_rho_trans = 0.25,
```

```

    metrop_var_phi1_trans = 1.00,
    metrop_var_phi2_trans = 1.00,
    metrop_var_A11_trans = 0.03,
    metrop_var_A22_trans = 0.30,
    interaction_indicator = 1,
    likelihood_indicator = 3,
    pair_id = pair_id)

```

```

## Progress: 10%
## lambda Acceptance (min): 14%
## lambda Acceptance (max): 61%
## rho Acceptance: 38%
## A11 Acceptance: 33%
## A22 Acceptance: 35%
## phi1 Acceptance: 42%
## phi2 Acceptance: 45%
## *****
## Progress: 20%
## lambda Acceptance (min): 12%
## lambda Acceptance (max): 58%
## rho Acceptance: 35%
## A11 Acceptance: 33%
## A22 Acceptance: 33%
## phi1 Acceptance: 42%
## phi2 Acceptance: 45%
## *****
## Progress: 30%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 55%
## rho Acceptance: 33%
## A11 Acceptance: 33%
## A22 Acceptance: 39%
## phi1 Acceptance: 43%
## phi2 Acceptance: 44%
## *****
## Progress: 40%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 53%
## rho Acceptance: 32%
## A11 Acceptance: 32%
## A22 Acceptance: 41%
## phi1 Acceptance: 43%
## phi2 Acceptance: 45%
## *****
## Progress: 50%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 54%
## rho Acceptance: 32%
## A11 Acceptance: 32%
## A22 Acceptance: 42%
## phi1 Acceptance: 43%
## phi2 Acceptance: 45%
## *****
## Progress: 60%

```

```

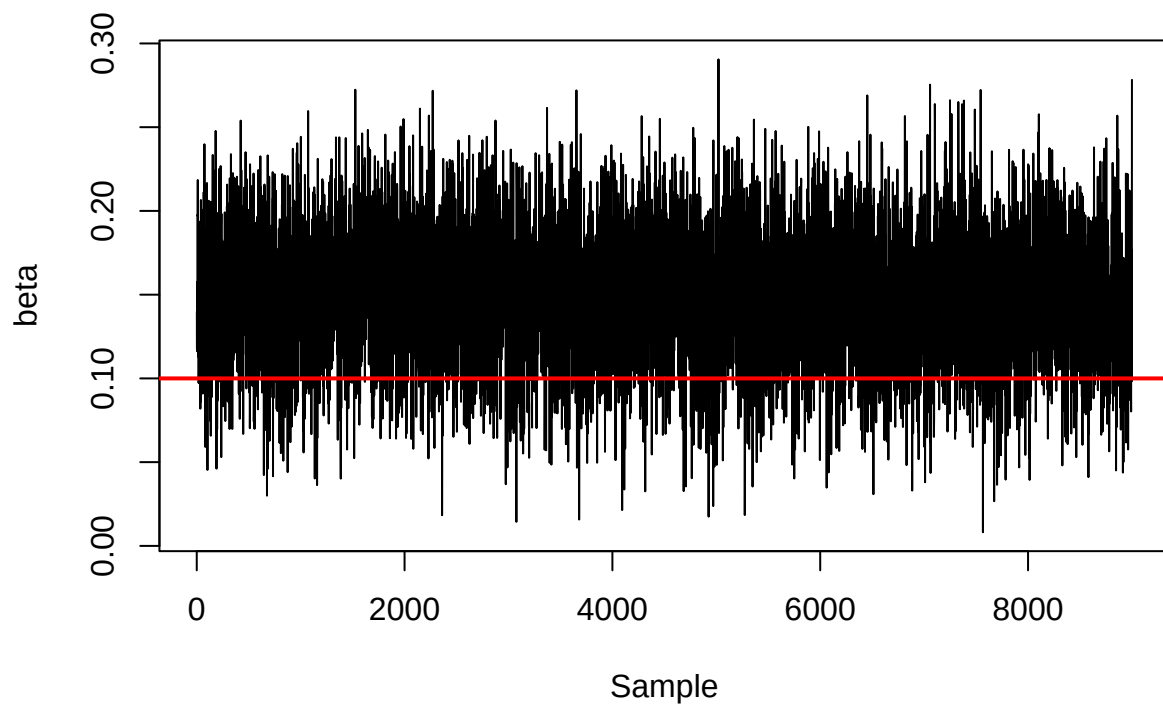
## lambda Acceptance (min): 10%
## lambda Acceptance (max): 54%
## rho Acceptance: 33%
## A11 Acceptance: 32%
## A22 Acceptance: 44%
## phi1 Acceptance: 42%
## phi2 Acceptance: 45%
## *****
## Progress: 70%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 55%
## rho Acceptance: 33%
## A11 Acceptance: 32%
## A22 Acceptance: 43%
## phi1 Acceptance: 42%
## phi2 Acceptance: 44%
## *****
## Progress: 80%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 54%
## rho Acceptance: 33%
## A11 Acceptance: 32%
## A22 Acceptance: 42%
## phi1 Acceptance: 42%
## phi2 Acceptance: 44%
## *****
## Progress: 90%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 53%
## rho Acceptance: 33%
## A11 Acceptance: 32%
## A22 Acceptance: 42%
## phi1 Acceptance: 42%
## phi2 Acceptance: 45%
## *****
## Progress: 100%
## lambda Acceptance (min): 11%
## lambda Acceptance (max): 53%
## rho Acceptance: 32%
## A11 Acceptance: 31%
## A22 Acceptance: 43%
## phi1 Acceptance: 42%
## phi2 Acceptance: 45%
## *****

```

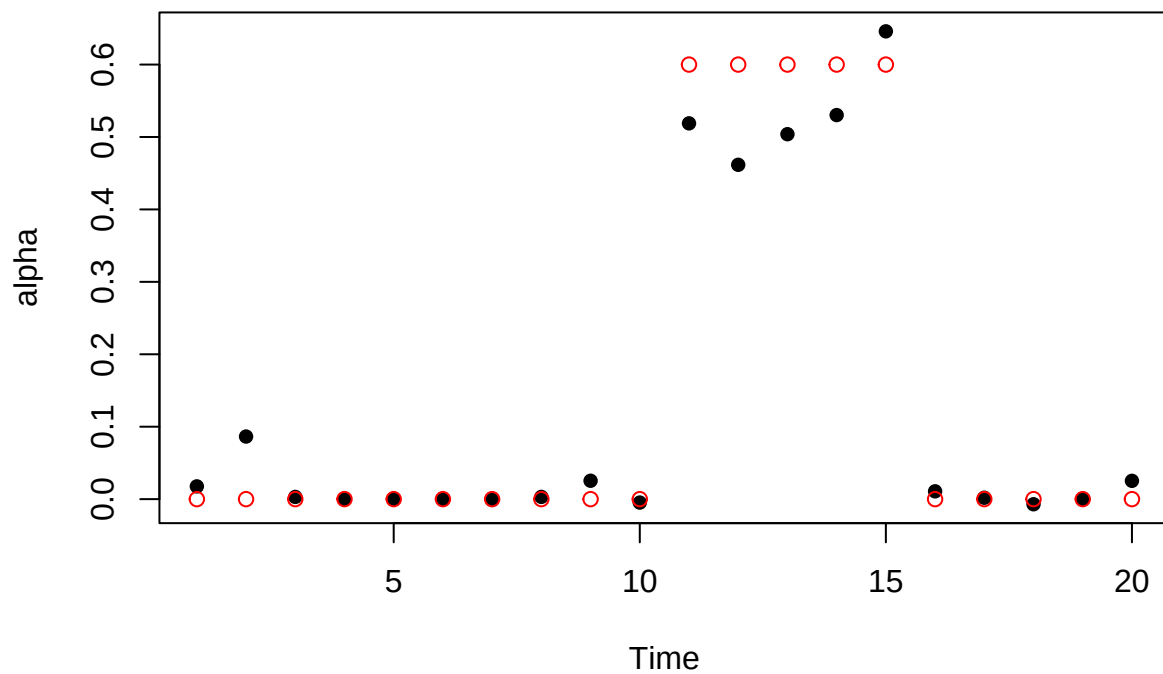
```

plot(results$beta[1, 1001:10000],
      type="l",
      ylab="beta",
      xlab="Sample")
abline(h=beta_true[2],
       col="red",
       lwd=2) #True value

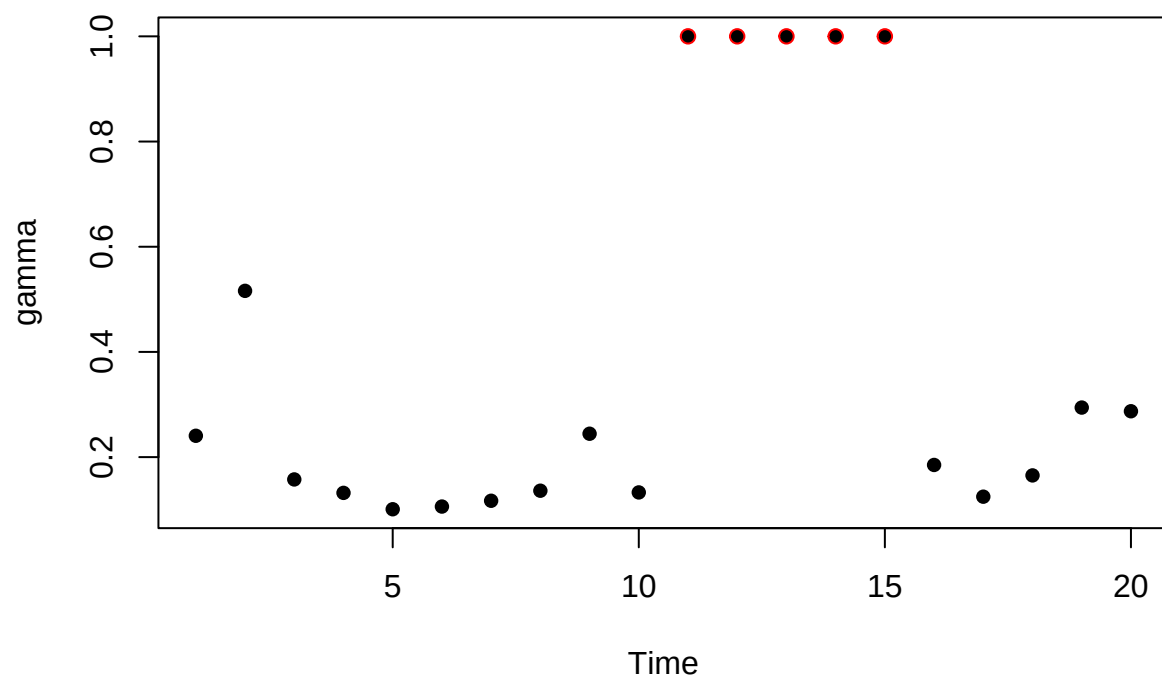
```



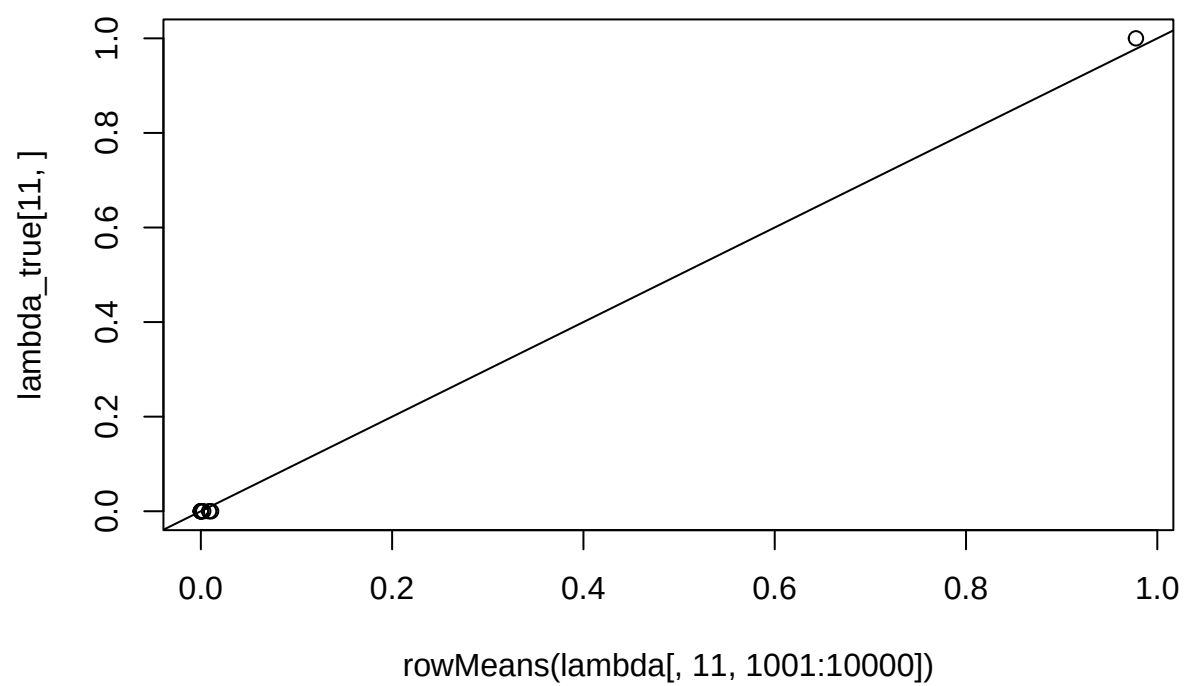
```
plot(rowMeans(results$alpha[,1001:10000]),  
      pch=16,  
      ylab="alpha",  
      xlab="Time")  
points(alpha_true,  
       col="red")
```



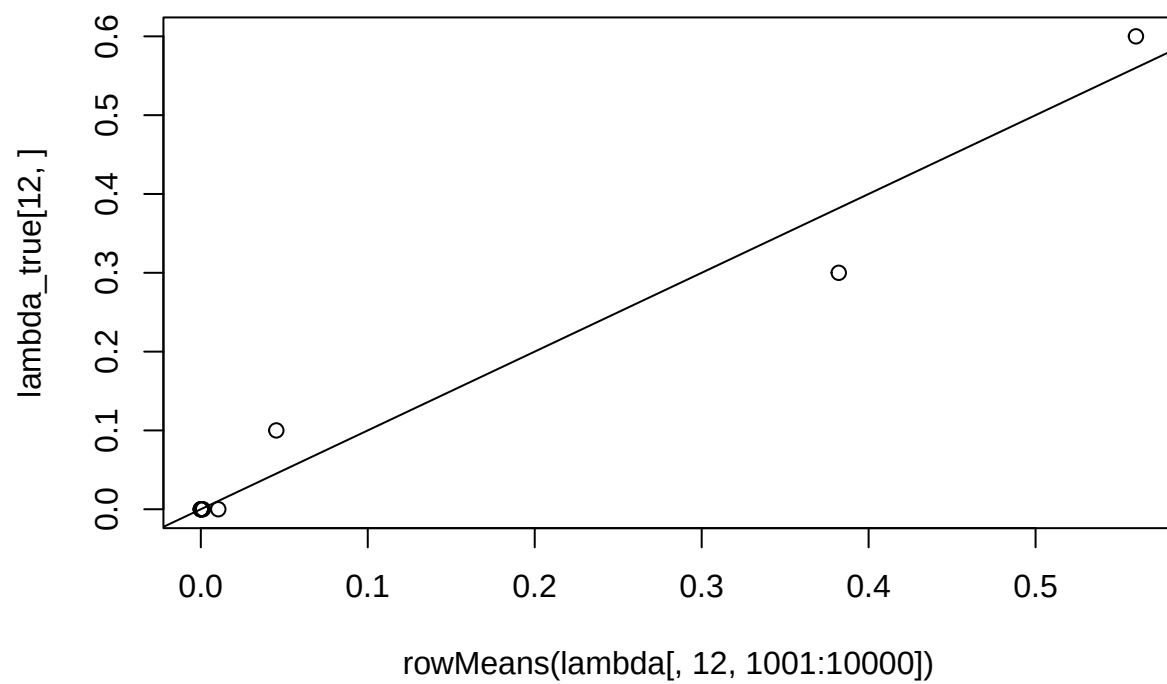
```
plot(rowMeans(results$gamma[,1001:10000]),  
     pch=16,  
     ylab="gamma",  
     xlab="Time")  
points(gamma_true,  
       col="red")
```



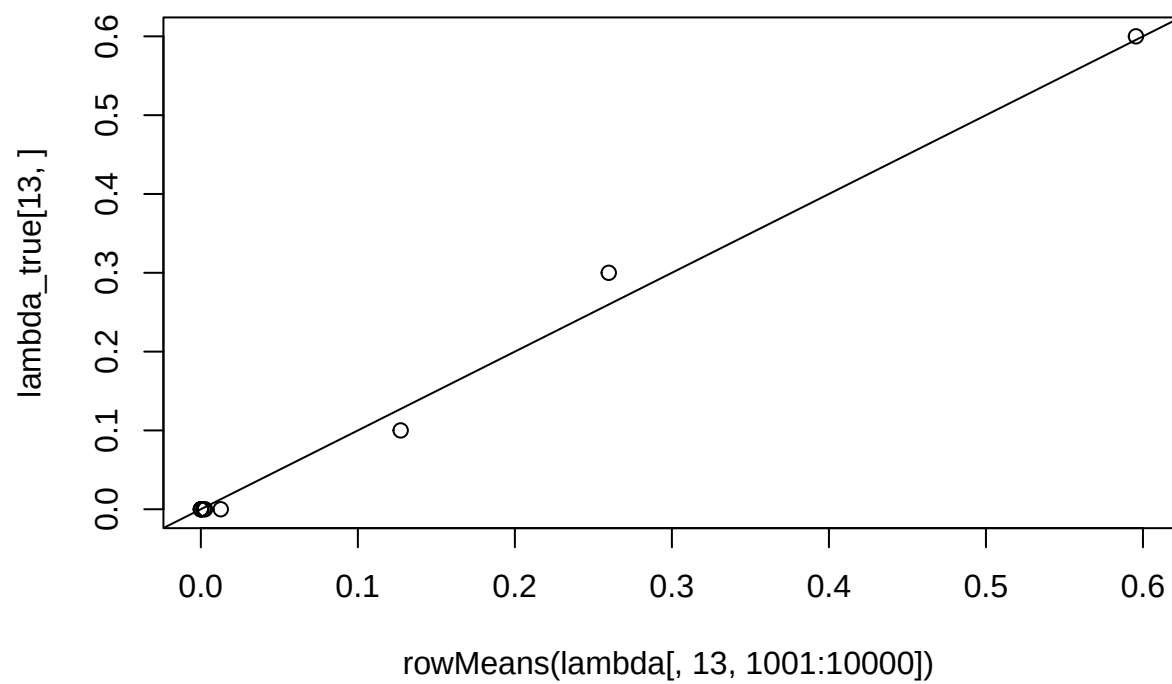
```
lambda<-array(as.numeric(unlist(results$lambda)),
              dim = c((q*(q+1)/2),
                      m,
                      10000))
plot(rowMeans(lambda[,11,1001:10000]),
     lambda_true[11,])
abline(0,1)
```

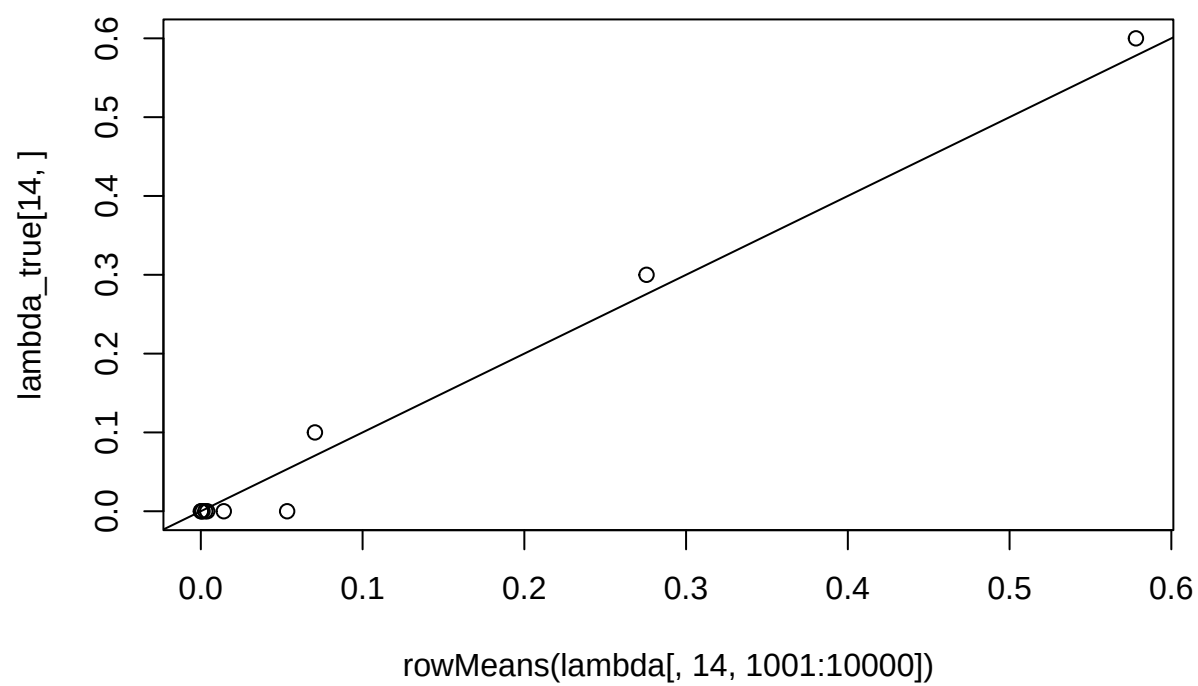
```
plot(rowMeans(lambda[, 12, 1001:10000]),  
     lambda_true[12,])  
abline(0,1)
```



```
plot(rowMeans(lambda[, 13, 1001:10000]),  
     lambda_true[13,])  
abline(0,1)
```



```
plot(rowMeans(lambda[,14,1001:10000]),  
     lambda_true[14,])  
abline(0,1)
```



```
plot(rowMeans(lambda[, 15, 1001:10000]),  
     lambda_true[15,])  
abline(0,1)
```

